

Payout Router

Выбирает, через какого партнёра отправить выплату. И объясняет почему.

0

ошибок в проверке
организаторов

9 942 17

заявки в секунду
в Docker с YJIT

сценариев под нагрузкой —
ни одной потерянной заявки



Выплата ищет партнёра

Деньги уходят через одного из трёх внешних провайдеров. Если никто не подходит — через свой запасной канал.

Нельзя

- Провайдер выключен
- Не работает с банком получателя
- Сумма вне его диапазона
- Дневной лимит выбран
- Слишком много выплат в обработке

Хотим

- vipay — 40% заявок, payflow — 35%, quickpay — 25%
- payflow прогоняет не меньше 2 млн ₽ в сутки
- У кого выплаты проходят чаще — тому больше

Договорённости спорят между собой. Роутер выбирает и показывает, чем пожертвовал.



Два этапа и запасной ход

Ruby 4.0

без нейросетей

14**жёстких правил**

Можно ли отдать заявку этому провайдеру. Да или нет.

13**целей с весами**

Кто из допущенных лучше. Считаем баллы.

**отправка**

Пока провайдер не ответил, его лимит и реквизит заняты.

**отказ —
следующий**

Отказал или молчит — идём к следующему по баллам.

**свой канал**

Когда внешних не осталось.

Правила и цели — разные слои. Правило отвечает «можно?», цель — «насколько хотим». Новое правило: один класс и строка в YAML.

inputs

constraints

strategies

scoring

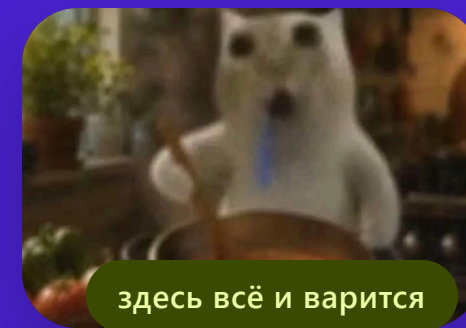
routing

state

analytics

output

server



здесь всё и варится

Каждый слой — своя папка. Состояние меняется только в state, остальное неизменяемо.

Почему выбран vipay: одна команда

```

payout_router — explain

$ payout_router explain op_105
op_105 · 25 000 ₽ · tinkoff · 2026-07-30 09:07:00
- payflow    bank_not_in_list    банк получателя не в списке
  tinkoff not in banks [sberbank, alfa]
+ vipay      best_score          лучший суммарный скор
  score 0.775; decisive vs quickpay: traffic_share (+0.3)
  traffic_share 0.650 → 1.000 × 0.30 = 0.3000  share 25.0% vs target 40.0%
  conversion    0.798 → 1.000 × 0.10 = 0.1000  0.87 → calibrated 0.798
  load          0.357 → 0.000 × 0.15 = 0.0000  utilization 64.3%
  ...
- quickpay    lower_score        допустим, но уступил по скору
  score 0.225 < 0.775 (vipay)

=> vipay: approved за 38 с

```

- Минус — провайдер выбыл. Рядом причина и число.
- Плюс — победитель. Ниже — из чего сложились баллы: оценка цели, место среди кандидатов, вес.
- decisive — чем именно победитель обошёл второго. Здесь vipay недобирает свою долю, и это перевесило.

Те же коды причин

в терминале, в файле решений и в отчёте: `bank_not_in_list · lower_score · provider_timeout · fallback_self_provider`. Ответ про одного провайдера — флаг `--why-not payflow`

Провайдер молчит — заявка идёт дальше



payout_router — explain · conversion

```
$ payout_router explain op_107 --simulation conversion --seed 46
op_107 · 800 ₽ · sberbank · 2026-07-30 09:08:00
- vipay          amount_below_minimum    800 < limit_amount_min 1000
- quickpay       amount_below_minimum    800 < limit_amount_min 1000
- payflow        provider_timeout        не ответил вовремя — идём к следующему
  dispatched, simulated result: expired after 631s
+ spacepayments  fallback_self_provider  внешних не осталось — self-provider

=> spacepayments: approved за 15 с (fallback), после 1 неудачной попытки
```

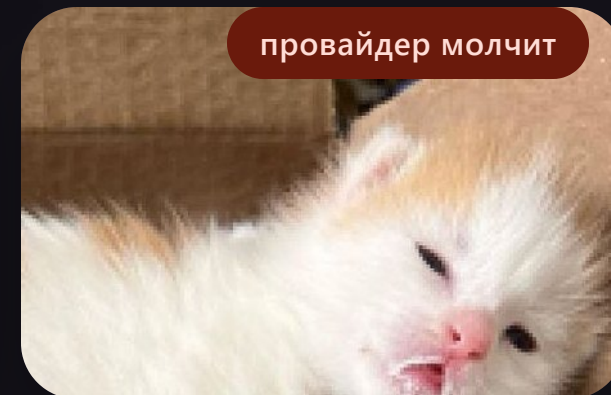
- Каждая попытка записана: кто, почему, чем кончилось.
- Пока провайдер молчит, его лимит и реквизит заняты. Ответ приходит позже — мы это учитываем.
- Рядом со сдачей лежит `routing_cascade_demo` с такими же цепочками.



payout_router — explain · conversion

```
$ payout_router explain op_106 --simulation conversion --seed 46
- payflow        amount_exceeds_limit    52000 > limit_amount_max 50000
- vipay          provider_timeout        expired after 567s
+ quickpay       fallback_after_failure  выбран после отказа предыдущего

=> quickpay: approved за 29 с
```



Цели по долям работают. Проверили.

0,0

промах по долям, пунктов
цели по долям включены

68,7

промах по долям, пунктов
цели по долям выключены

2 000 заявок, где все три
провайдера подходят под каждую.

Включили цели по долям — попали в 40 / 35 /
25 точно. Выключили — промах 68,7 пункта.

payout_router — stress

```
$ payout_router stress --scenario open_field open_field_no_shares
```

сценарий	заявок	с_выбором%	без_маршрута	fallback%	Σ откл	инварианты
open_field	2000	100.0	0	0.0	0.0	ок
open_field_no_shares	2000	100.0	0	0.0	68.7	ок

На обычных данных выбора почти нет: у 98% заявок после правил остаётся один кандидат. Поэтому вклад стратегии меряем на очереди с выбором.

Одна строка в YAML — другой баланс



config/policy.yml

```
goals:
  traffic_share: 0.30 # доля по числу заявок
  volume_share: 0.10 # доля по объёму
  cascade_priority: 0.10 # очередь в каскаде
  amount_band: 0.10 # диапазон суммы
  conversion: 0.10 # конверсия
  bank_affinity: 0.05 # конверсия по банку
  load: 0.15 # загрузка лимитов
  turnover_min: 0.05 # обязательство по обороту
  rate_headroom: 0.05 # запас по интенсивности
```

Новый провайдер — запись в providers.json

Проверили: четвёртый провайдер попал в роутинг и в отчёт без правки кода.

ПОЛИТИКА	ПРОМАХ ПО ДОЛЯМ, П.П.	ОДОБРЕНИЙ ИЗ 10
balanced — сдаём	10,0	6,54
conversion_first	70,0	7,19
turnover_pressure	110,0	6,65
profit_first	130,0	6,26

Первым и по долям, и по одобрениям быть нельзя: это разные цели. Роутер показывает цену каждого варианта, выбирает бизнес.

Промех — сумма отклонений от обещанных долей в процентных пунктах. Ожидаемые одобрения — по конверсии из истории операций. Всего в config/policies/ лежит десять готовых политик.

Отчёт называет параметр и число

Распределение факт против цели из снимка и против достижимой

Достижимая цель — доля, которую провайдер получил бы при идеально пропорциональном роутинге среди тех заявок, где он вообще проходит hard-правила. Отклонение от неё показывает работу стратегии; отклонение от цели из снимка — ещё и то, насколько сама цель выполнима на этих данных.

ПРОВАЙДЕР	ДОЛЯ ПО ЗАЯВКАМ	ЗАЯВОК	ФАКТ	ЦЕЛЬ	ОТКЛ.	ДОСТИЖИМО	ОТКЛ. ОТ ДОСТ.	ОБЪЁМ	ДОЛЯ ОБЪЁМА / ЦЕЛЬ
vipay		4	40.0%	40%	+0.0	22.5%	+17.5	102 000 ₽	26.4% / 50%
payflow		3	30.0%	35%	-5.0	25.2%	+4.8	88 800 ₽	23.0% / 30%
quickpay		3	30.0%	25%	+5.0	52.4%	-22.4	195 000 ₽	50.5% / 20%

КРИТИЧНО · DAILY_LIMIT_PRESSURE · TRAFFIC_PERCENTAGE: 35 → 20

payflow: дневной лимит использован на 99.6% (2 988 800 ₽ из 3 000 000 ₽) — снизить traffic_percentage с 35 до 20 или поднять daily_amount_limit до 3 800 000 ₽

- Рядом с обещанной долей — достижимая: сколько провайдер мог получить на этих данных.
- 13 правил рекомендаций. Каждое называет параметр и новое значение.

Отдаёт решение за миллисекунды

9 942

заявки в секунду в Docker: 50 000 за 5,0 с

254

автотеста, покрытие 96%

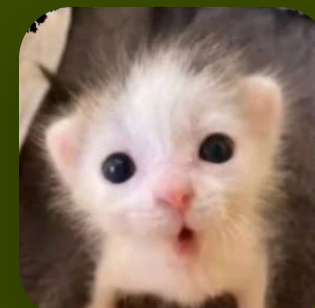
17 · 15 990 · 0

сценариев под нагрузкой · заявок · потерь

```
$ payout_router serve --port 8090
PayoutRouter слушает http://127.0.0.1:8090 -
  POST /route (заявка или массив), GET /report, /state, /metrics, /health

$ curl -X POST 127.0.0.1:8090/route -d '{
  "operation": { "operation_id": "op_1", "amount": 15000, "bank": "sberbank" },
  "providers": [{ "payment_system": "vipay", "status": "inactive" }] }'
{
  "operation_id": "op_1",
  "selected_provider": "payflow",
  "attempts": [
    { "provider": "vipay", "decision": "skipped", "reason": "provider_inactive" },
    { "provider": "payflow", "decision": "selected", "reason": "best_score" },
    { "provider": "quickpay", "decision": "skipped", "reason": "lower_score" }
  ],
  "simulated_result": "approved", "latency_sec": 52
}
```

Состояние провайдеров можно прислать вместе с заявкой: источником правды остаётся ваша система.



шесть тысяч в секунду

Знаем, насколько близки к идеалу



payout_router — bound

\$ payout_router bound

назначение	одобрений	комментарий
оптимум	7.8	знает всю очередь заранее
наш роутинг	6.54	онлайн, по одной заявке
худшее допустимое	5.44	нижняя точка отсчёта
— оптимум, если ещё держать доли	6.72	другая задача: никто выше своей доли

Онлайн-роутинг взял 83.8% от точного оптимума этой очереди.

Из отставания в 1.26 одобрения 1.09 — цена обязательства держать целевые доли, и 0.18 — цена решений по одной заявке.

83,8%

от точного оптимума

на публичной очереди; на тысяче заявок — 96,4%

Отставание разложено

1,09 одобрения — цена обязательства держать доли партнёров. 0,18 — цена того, что решаем по одной заявке, не зная следующих.

Оптимум считается точным min-cost flow по всей очереди сразу и лежит в каждом отчёте блоком `optimality`.

Наше распределение допустимо в задаче эталона — значит это настоящая верхняя граница, накрутить её нельзя.

Чем отличаемся

Каждое решение объяснено числами

Видно, чем именно победитель обошёл второго. В отчёте — примеры со ссылкой на метод в коде.

Честная модель времени

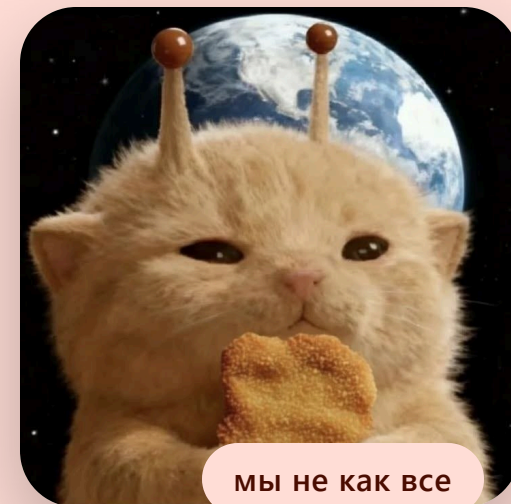
Лимит занят, пока провайдер не ответил. 17 сценариев под нагрузкой нашли три ошибки — исправлены.

Стратегия доказана числами

Промах по долям 0,0 с целями против 68,7 без них — там, где выбор есть в каждой заявке.

Готово к запуску

HTTP-сервис, метрики Prometheus, Docker, 9 942 заявки в секунду. Всё на Ruby, без нейросетей.



мы не как все

Каждый пункт ТЗ закрыт, и у каждого есть команда, которой это проверяется за минуту.

`explain op_101` `strategies · compare` `routing_decisions.json` `routing_report.json` `routing_cascade_demo.json` `rake stress` `rake submit`

полная сверка — `docs/criteria.md`

Задавайте вопросы

Любой ответ можно проверить в терминале прямо сейчас.



Код, отчёты и эта презентация
[github.com/
Mojarung/hack_genesis_2026](https://github.com/Mojarung/hack_genesis_2026)

MISIS MOJARUNG

